HiddenGem

Personas, Storyboards & Stretch Features

Updated design: follow/unfollow social graph moved to stretch feature

Prepared by: Cheryl Wang Smith
Course: SWDV 691: SOFTWARE DEVELOPMENT CAPSTONE 1W 26/SP2
Date: March 29, 2026

What Changed From the Original Design

The following changes were made in response to instructor feedback and scope review. The core discovery-and-sharing experience is unchanged. The social graph (follow/unfollow) has been moved to a stretch feature to keep the MVP tightly scoped.

What changed	Detail
Follow / unfollow social graph	Moved to stretch feature. Community page, follower/following lists, and personalized "Following" feed filter are all deferred. The core app works fully without them.
Persona updates	Maya and Sofia no longer mention following other users as a goal -- their discovery flow is purely map-and-filter based. James's goals updated to remove "build a follower base." A new stretch persona (Alex) is added to represent the social follow use case.
Storyboard updates	Flow 3 (Following Another Explorer) removed from MVP. A stretch storyboard is added showing Alex's follow flow.
MVP feature table	Community page and follow/unfollow endpoints moved out of MVP feature set.
Database naming	All table names corrected to singular (user, gem, photo, tag, comment, save, follow). All primary keys corrected to tableNameID convention (userID, gemID, etc.) per instructor feedback.

Section 1: MVP User Personas

These three personas represent the types of people served by the MVP. They drive all design, feature, and prioritization decisions for the minimum viable product.

Persona 1: Maya Chen

Maya Chen The Urban Explorer | Age 28	
Background	Maya recently relocated to a new city for work. She loves finding unique spots but finds mainstream apps only surface tourist traps and chain restaurants. She spends weekends deliberately exploring neighborhoods on foot.
Goals	Discover authentic, non-commercial local spots. Build a personal saved list of favorite places. Feel connected to the community of her new city.
Pain Points	Mainstream maps bury unique locations under commercial results. Social media posts are hard to search by geography. She has no trusted local friends yet to ask for recommendations.
"I want to find the spots that feel like secrets, not the ones with 5,000 reviews on Yelp."	

Persona 2: James Okafor

James Okafor The Local Sharer | Age 42	
Background	James has lived in the same neighborhood for 15 years. He is deeply proud of his community and knows every interesting corner of it. He regularly tells friends about hidden spots but has no platform to share them more broadly.
Goals	Share his local knowledge with people who would truly appreciate it. Contribute to his community. Leave a record of places he loves before they disappear.
Pain Points	No easy way to share location-specific knowledge beyond texting friends one at a time. Mainstream apps reward commercial businesses, not community character.
"I know places in this neighborhood that most people drive right past. I just need a way to show people they exist."	

Persona 3: Sofia Ramirez

Sofia Ramirez

The Weekend Adventurer | Age 34

Background	Sofia is a busy professional who plans weekend outings with her partner. She wants experiences that feel special, not the same destinations everyone else visits. She is willing to travel up to an hour for a truly unique spot.
Goals	Plan memorable weekend outings efficiently. Find places that feel curated and personal, not generic. Avoid disappointment from over-hyped destinations.
Pain Points	Spends too much time researching across multiple platforms with inconsistent results. Hard to know if a place is still active or genuinely worth the drive.

"I just want someone to tell me one genuinely great thing within 45 minutes of home that I have not already seen."

Section 2: MVP Storyboards

Each storyboard describes how a persona achieves their goal through the application, step by step, showing the page they are on, what they do, and how the UI responds. Wireframe annotations reference these storyboards.

Storyboard 1: Discovering a Hidden Gem (Maya)

Scenario: Maya is new to the city and wants to find a genuinely interesting nature spot for her weekend. She has heard about HiddenGem from a co-worker and opens it for the first time.

Step	Page	User action & UI response
1	**Welcome / Landing (/)**	Maya lands on the marketing page. She reads the tagline "Discover Local" and sees the feature highlights. She clicks "Get Started Free".
2	**Sign Up (/signup)**	Maya enters her full name, email, and password. She checks the Terms of Service box and clicks "Create Account". The API creates her user row with is_verified = FALSE and sends a verification email.
3	**Email Verification (/verify)**	Maya sees the "Check Your Email" screen. She opens her inbox, clicks the link, and is redirected back. Her account is now verified (is_verified = TRUE).
4	**Discover -- Feed (/discover)**	Maya lands on the Discover page in Feed view. She sees a card grid of "Hidden Gems Near You". She types "ABC District" in the location search sidebar.
5	**Discover -- Feed (filtered)**	The feed updates to show gems near ABC District. Maya clicks the #nature tag filter pill. The feed narrows to show only nature category gems with matching tags.
6	**Discover -- Map view**	Maya clicks the Map toggle. The same filtered results now show as colored pins on a Google Maps panel. She can see where the gems are relative to each other.
7	**Gem Detail (/gems/:id)**	Maya clicks the "Secret Waterfall Trail" pin. A sidebar preview appears showing the cover photo and save count. She clicks through to the full detail page.
8	**Gem Detail -- saved**	Maya reads the full description, scrolls through 2 photos, and reads the comments. She clicks the bookmark icon. The API inserts a row in the save table. The save count increments from 24 to 25.

Storyboard 2: Sharing a Hidden Gem (James)

Scenario: James discovers a beautiful old mosaic wall he has walked past for years and has never seen anyone photograph. He wants to share it with the HiddenGem community before it gets demolished.

Step	Page	User action & UI response
1	**Sign In (/signin)**	James enters his email and password. He clicks "Sign In". The API validates his credentials and returns a JWT. He is redirected to the Discover page.
2	**Create Gem (/create)**	James clicks "Create" in the top navigation bar. The submission form opens.
3	**Create -- photo upload**	James drags a photo from his phone onto the upload zone. The API sends it to Cloudinary. A thumbnail preview appears in the form. He uploads a second photo.
4	**Create -- location**	James clicks the current-location icon. The form auto-fills "Calle 24, Mission District" as the location label and sets the coordinates. He sees a small map confirming the pin placement.
5	**Create -- details**	James types the gem name "1970s Tile Mosaic -- Calle 24" and writes a description. He adds tags: #art, #urban, #historic. He sets privacy to Public.
6	**Create -- submit**	James clicks "Publish Hidden Gem". The API validates input, stores the photos via Cloudinary, inserts the gem row into the database, and inserts tag rows. He is redirected to the new gem's detail page.
7	**Gem Detail (/gems/:id)**	James sees his new gem live. The cover photo, description, and tags are all displayed. He copies the URL from his browser and shares it in a neighborhood group chat.

Storyboard 3: Saving and Commenting (Sofia)

Scenario: Sofia is planning a Sunday outing and wants to find an art or food gem within 30 minutes of home that she can visit with her partner.

Step	Page	User action & UI response
1	**Discover -- Feed (/discover)**	Sofia opens HiddenGem. She is already logged in. She sets location search to her home neighborhood and sets the distance filter to 30 km.
2	**Discover -- filtered**	Sofia clicks the #food tag pill. The feed shows 8 nearby food gems. She scans the card grid, comparing cover photos and save counts.
3	**Gem Detail (/gems/:id)**	Sofia clicks the "Secret Speakeasy Cafe" card. The detail page loads with 3 photos, a full description including opening hours, and 14 comments from other visitors.

4	**Gem Detail -- comments**	Sofia reads the comments. She sees one that says "best visited on a Tuesday, ask for the back room." She clicks Post Comment and adds: "Going this Sunday! Thanks for the tip."
5	**Gem Detail -- save**	Sofia clicks the bookmark icon. The gem is saved to her personal list. Save count increases to 31.
6	**User Profile -- Saved tab**	Sofia navigates to her profile and clicks the Saved tab. She sees "Secret Speakeasy Cafe" at the top of her saved list, sorted by most recently saved.

Section 3: Updated MVP Feature Set

This is the confirmed MVP scope after moving follow/unfollow to stretch. Every feature below must be completed before the follow stretch feature is started.

Authentication & onboarding	
Register	Email, password, display name. JWT returned on success.
Email verification	"Check Your Email" screen. Resend option. Blocks gem submission until verified.
Sign In / Sign Out	Email + password. JWT-based session.
User profile page	Shows submitted gems (My Gems tab) and saved gems (Saved tab).

Discover -- feed & map	
Feed view (card grid)	Default view. Cards show cover photo, author, title, description excerpt, location, tags, save count.
Map view	Google Maps panel. Pins colored by category. Click pin to open sidebar preview.
View toggle	Feed / Map / Share toggle inline at top of Discover page.
Location search	Search by city or neighborhood. Distance filter in sidebar.
Tag filter	Popular tag pills in sidebar. Clicking filters both feed and map in real-time.
Sort	Newest / nearest / most saved sort dropdown.
Recent activity sidebar	Shows recent community activity: new gems, likes.

Gem detail page	
Full content	All photos (carousel), full description, author info with avatar, location label.
Action bar	Like count, comment count, share button, save/bookmark button, flag button.
Comment thread	Time-ordered. Post new comment with text input. Pagination ("Load more"). Author badge on submitter replies.

Create & edit gem	
Create form	Photo upload (drag-and-drop, up to 3, 10MB, PNG/JPG/GIF). Location picker with current-location detection. Description with 300-char limit. Tags. Privacy (Public / Private).
Publish / draft	"Publish Hidden Gem" primary button. "Save as Draft" secondary.

Edit form	Pre-filled. Change photo, update any field. Delete Post (destructive). Post statistics panel (Views, Likes, Comments).

Saves & moderation	
Save / unsave	Bookmark any gem. Saved gems appear in profile Saved tab sorted by most recent.
Flag	Flag button on gem detail action bar. Flagged gems hidden from public map pending review.

Section 4: Stretch Feature -- Follow / Unfollow Social Graph

The stretch feature is the follow/unfollow social graph. It is the natural next feature after the MVP because it transforms HiddenGem from a static map into a personalized, living feed. It adds the Community page, follower/following lists, and a personalized discovery feed filtered to gems from followed users.

The stretch feature aligns directly with the follow table in the revised database design and the follow endpoints in the service layer design.

Stretch Persona: Alex Rivera

Alex Rivera The Social Curator | Age 31	
Background	Alex is a food and travel blogger who uses HiddenGem to discover spots shared by a small circle of trusted local explorers. He follows 12 people whose taste he trusts completely and ignores the broader map. He contributes gems regularly and enjoys seeing his follower count grow.
Goals	Build a curated feed of gems from people he trusts. Grow his own audience of followers who value his taste. See new gems from followed users without having to search the full map.
Pain Points	The main Discover feed surfaces too many low-quality gems. He wants a filtered view that shows only what the people he follows have posted. Currently has to manually visit each trusted user's profile to check for new posts.
"I follow 12 people on here and I trust all of them. I just want a feed that shows me what they've found this week."	

Stretch Storyboard: Following Another Explorer (Alex)

Scenario: Alex wants to follow James, whose gem quality he has noticed after discovering three great spots James submitted. He then wants to see James's new gems in his personalized feed.

Step	Page	User action & UI response
1	Gem Detail (/gems/:id)	Alex reads a gem submitted by @jamesokafor and is impressed. He clicks the author's display name or avatar to visit James's profile.

2	**User Profile (/profile/jamesokafor)**	Alex sees James's profile: 127 gems, 1.2K followers. He sees the "Follow" button (outline pill). He clicks it.
3	**User Profile -- following**	The API inserts a row in the follow table (followerID = Alex, followingID = James). The "Follow" button transitions to a filled green "Following" pill. James's follower count increments to 1,201.
4	**Discover -- Following feed**	Alex navigates to the Discover page. He clicks the "Following" feed filter. The feed now shows only gems from the 12 people he follows, including James's latest submission.
5	**Community (/community)**	Alex navigates to Community. He sees two tabs: "Followers" (showing who follows him) and "Following" (showing James and 11 others). Each card shows bio, gem count, follower count, and latest gem preview.
6	**Community -- unfollow**	Alex changes his mind about one user. He clicks their "Following" pill. A confirmation appears. He confirms, and the API deletes the follow row. The button reverts to "Follow". His following count drops by 1.

Stretch Feature Set

Follow / unfollow social graph (stretch)	
Community page	Two tabs: Followers and Following, each with a count. Search bar. User cards show avatar, name, handle, bio, gem count, follower count, join date, latest gem preview, Follow/Following button.
Follow / Unfollow	"Follow" (outline pill) transitions to "Following" (green pill) on click. Clicking "Following" again unfollows with a confirmation. Button state is a real-time primary-key lookup on the follow table.
Following feed filter	On the Discover page, a "Following" option in the feed/map view toggle. When active, only gems from followed users are shown (privacy != private).
Follower / following counts	Shown on user profile card. Aggregated at query time from the follow table; not stored as a denormalized column.
Profile follow button	When viewing another user's profile, the Follow/Following button appears. Hidden on own profile.

End